###data cleaning

Data Cleaning & Visualization Project Work on a raw dataset to clean, process, and visualize insights.

Key Features: Handle missing values, outliers, and duplicates Use Python libraries like Pandas, Matplotlib, or Seaborn Create dashboards or visual reports of key findings Expected Outcome: Learn data preprocessing, visualization, and storytelling with data

Titanic Data Cleaning Visualization Project

step1: importing libaries

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

- pandas = for data handaling
- numpy = for numerical operation
- matplotlib = for basic chart
- seaborn = for beautiful statistical plots
- warning = hides unnecessary warnings

Step2:Load Titanic Data Set

```
df = sns.load_dataset("titanic")
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 891,\n  \"fields\": [\
n    {\n      \"column\": \"survived\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"pclass\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 1,\n
\"max\": 3,\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          3,\n          1\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"sex\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \"female\",\n          \"male\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"age\",\n      \"properties\": {\n
```

```
\"dtype\": \"number\",\n        \"std\": 14.526497332334044,\n
\"min\": 0.42,\n        \"max\": 80.0,\n        \"num_unique_values\":
88,\n        \"samples\": [\n          0.75,\n          22.0\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sibsp\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1,\n        \"min\": 0,\n        \"max\": 8,\n
\"num_unique_values\": 7,\n        \"samples\": [\n          1,\n
0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"parch\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0,\n        \"min\": 0,\n        \"max\": 6,\n
\"num_unique_values\": 7,\n        \"samples\": [\n          0,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"fare\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 49.693428597180905,\n        \"min\": 0.0,\n        \"max\":
512.3292,\n        \"num_unique_values\": 248,\n        \"samples\":
[\n          11.2417,\n          51.8625\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"embarked\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
3,\n        \"samples\": [\n          \"S\",\n          \"C\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"class\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
3,\n        \"samples\": [\n          \"Third\",\n          \"First\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"who\",\n      \"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n          \"man\",\
n          \"woman\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"adult_male\",\n      \"properties\": {\n        \"dtype\":
\"boolean\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          false,\n          true\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"deck\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 7,\n
\"samples\": [\n          \"C\",\n          \"E\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"embark_town\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Southampton\",\n          \"Cherbourg\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"alive\",\n      \"properties\": {\
n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n
\"samples\": [\n          \"yes\",\n          \"no\"\n        ],\n
```

\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"alone\",\n   \"properties\": {\
n   \"dtype\": \"boolean\",\n   \"num_unique_values\": 2,\n
\"samples\": [\n   true,\n   false\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 8,\n \"fields\": [\n
{\n   \"column\": \"survived\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 314.8713661874558,\n
\"min\": 0.0,\n   \"max\": 891.0,\n   \"num_unique_values\":
5,\n   \"samples\": [\n   0.3838383838383838,\n
1.0,\n   0.4865924542648585\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"pclass\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\":
314.2523437079693,\n   \"min\": 0.8360712409770513,\n
\"max\": 891.0,\n   \"num_unique_values\": 6,\n
\"samples\": [\n   891.0,\n   2.308641975308642,\n
3.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"age\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 242.9056731818781,\n   \"min\": 0.42,\n   \"max\":

```
714.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
29.69911764705882,\n          28.0,\n          714.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sibsp\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 314.4908277465442,\n
\"min\": 0.0,\n        \"max\": 891.0,\n        \"num_unique_values\":
6,\n        \"samples\": [\n          891.0,\n
0.5230078563411896,\n          8.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"parch\",\n      \"properties\": {\
n        \"dtype\": \"number\",\n        \"std\": 314.65971717879,\n
\"min\": 0.0,\n        \"max\": 891.0,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          0.38159371492704824,\n
6.0,\n          0.8060572211299559\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"fare\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 330.6256632228577,\n
\"min\": 0.0,\n        \"max\": 891.0,\n        \"num_unique_values\":
8,\n        \"samples\": [\n          32.204207968574636,\n
14.4542,\n          891.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```


```
df.isnull().sum()
```

```
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

```
# Fill missing Age values with the median
df['age'] = df['age'].fillna(df['age'].median())

# Fill missing Embarked values with the mode
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# Fill missing Embark Town values with the mode
df['embark_town'] = df['embark_town'].fillna(df['embark_town'].mode()
```

```
[0])

# Drop the deck column
df = df.drop(columns=['deck'])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_12819/1565703120.py in <cell line: 0>()
      9
     10 # Drop the deck column
---> 11 df = df.drop(columns=['deck'])

/usr/local/lib/python3.12/dist-packages/pandas/core/frame.py in drop(self, labels, axis, index, columns, level, inplace, errors)
   5579                 weight  1.0     0.8
   5580         """
-> 5581         return super().drop(
   5582             labels=labels,
   5583             axis=axis,

/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py in drop(self, labels, axis, index, columns, level, inplace, errors)
   4786         for axis, labels in axes.items():
   4787             if labels is not None:
-> 4788                 obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4789
   4790         if inplace:

/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py in _drop_axis(self, labels, axis, level, errors, only_slice)
   4828                 new_axis = axis.drop(labels, level=level, errors=errors)
   4829             else:
-> 4830                 new_axis = axis.drop(labels, errors=errors)
   4831             indexer = axis.get_indexer(new_axis)
   4832

/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in drop(self, labels, errors)
   7068         if mask.any():
   7069             if errors != "ignore":
-> 7070                 raise KeyError(f"{labels[mask].tolist()} not found in axis")
   7071             indexer = indexer[~mask]
   7072         return self.delete(indexer)

KeyError: "['deck'] not found in axis"
```

```
df.columns
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'embark_town',
'alive',
       'alone'],
      dtype='object')
```

```
df.isnu11().sum()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call
last)
/tmp/ipykernel_12819/2976904316.py in <cell line: 0>()
----> 1 df.isnu11().sum()

/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py in
__getattr__(self, name)
   6297         ):
   6298             return self[name]
-> 6299         return object.__getattribute__(self, name)
   6300
   6301     @final

AttributeError: 'DataFrame' object has no attribute 'isnu11'
```

```
df.isnull().sum()
```

```
survived       0
pclass         0
sex            0
age            0
sibsp          0
parch          0
fare           0
embarked       0
class          0
who            0
adult_male     0
embark_town    0
alive          0
alone          0
dtype: int64
```

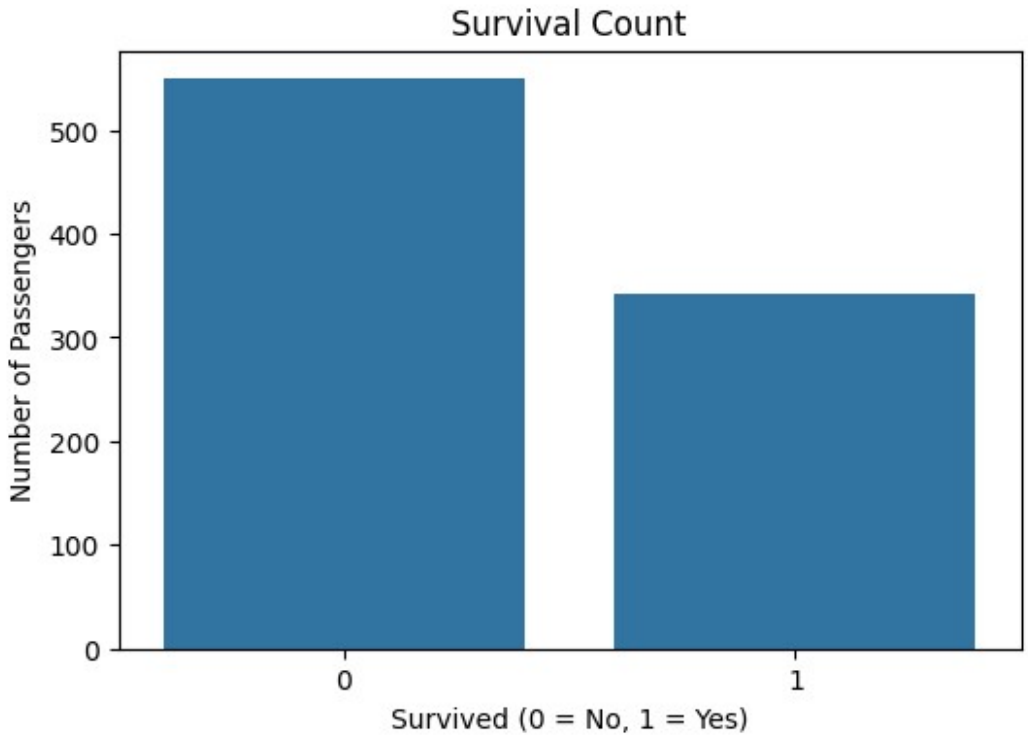
```
plt.figure(figsize=(6,4))

sns.countplot(x='survived', data=df)

plt.title("Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
```

```
plt.ylabel("Number of Passengers")

plt.show()
```



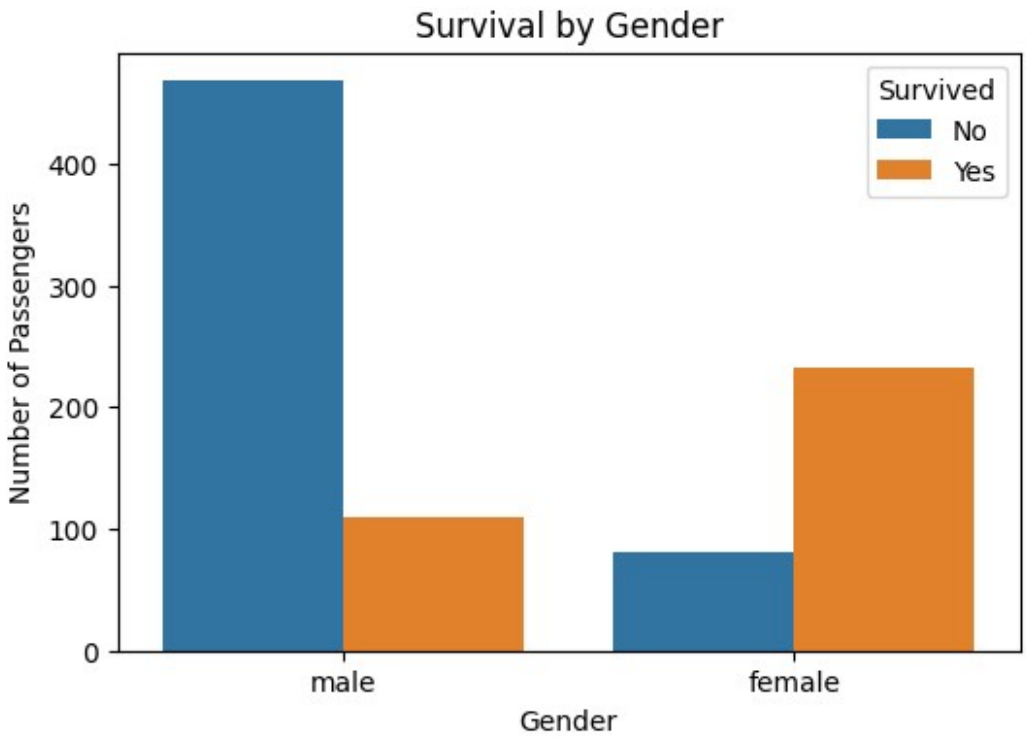

```
plt.figure(figsize=(6,4))

sns.countplot(x='sex', hue='survived', data=df)

plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")

plt.legend(title="Survived", labels=["No", "Yes"])

plt.show()
```



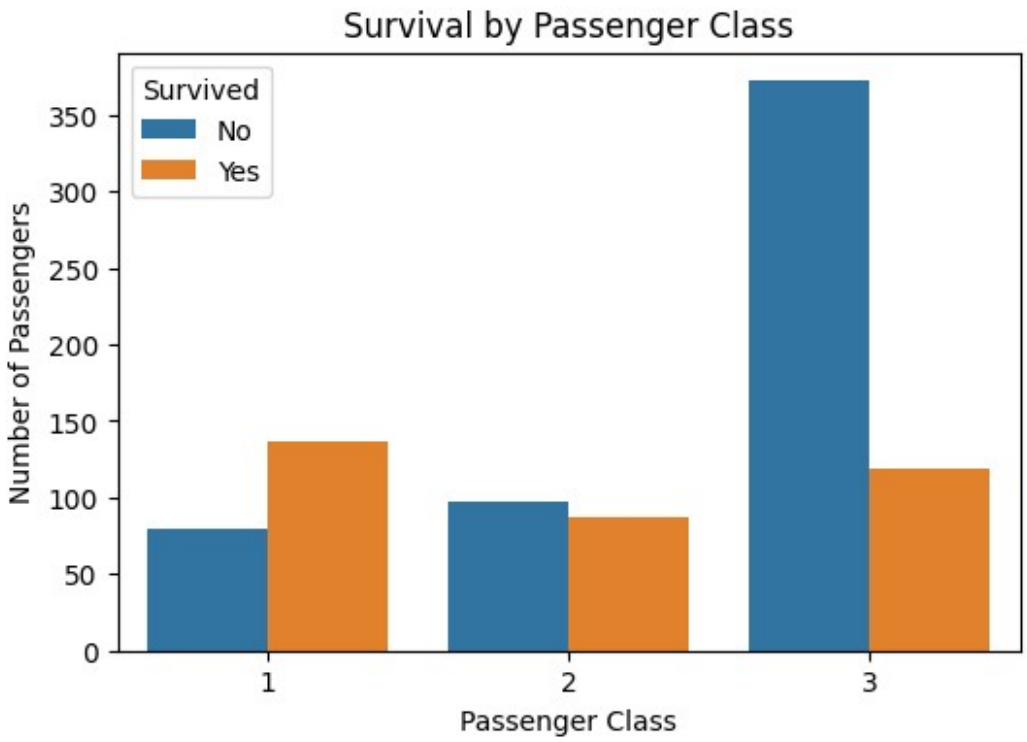

```
plt.figure(figsize=(6,4))

sns.countplot(x='pclass', hue='survived', data=df)

plt.title("Survival by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")

plt.legend(title="Survived", labels=["No", "Yes"])

plt.show()
```

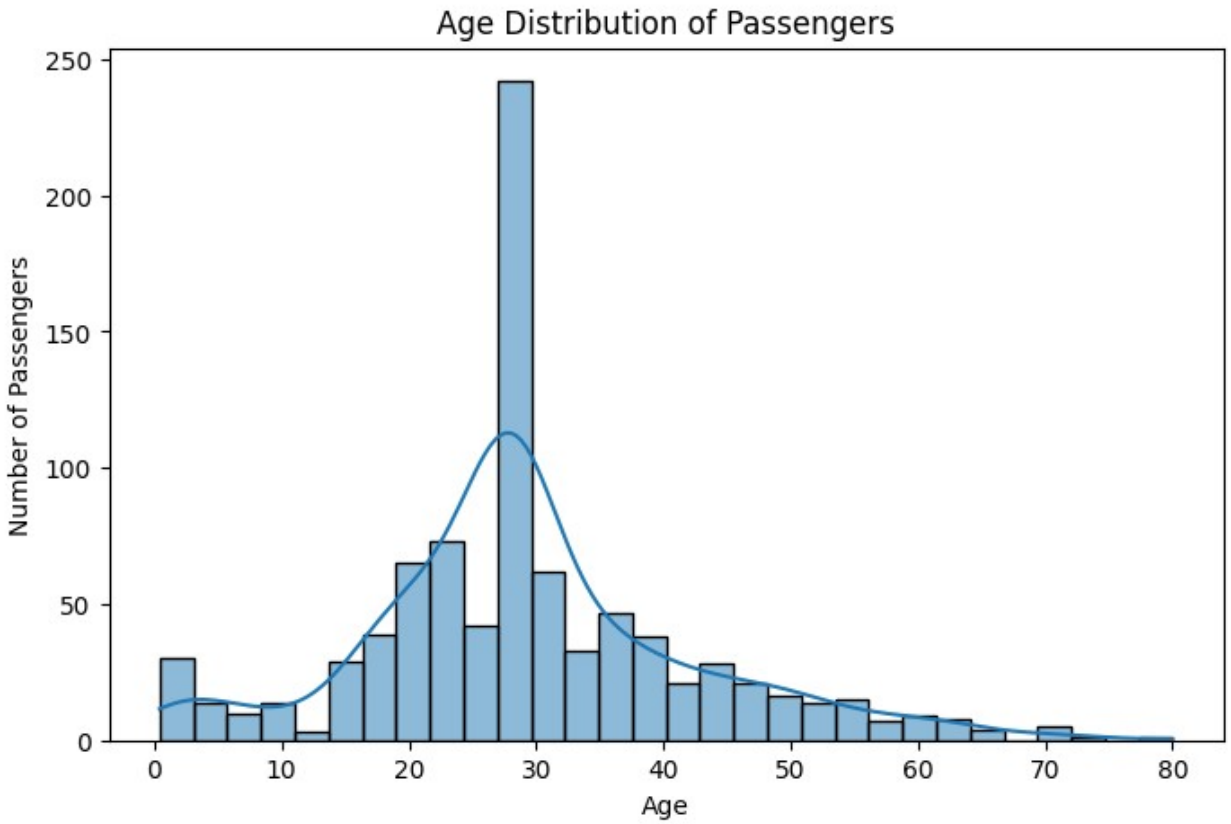



```
plt.figure(figsize=(8,5))

sns.histplot(df['age'], bins=30, kde=True)

plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Number of Passengers")

plt.show()
```



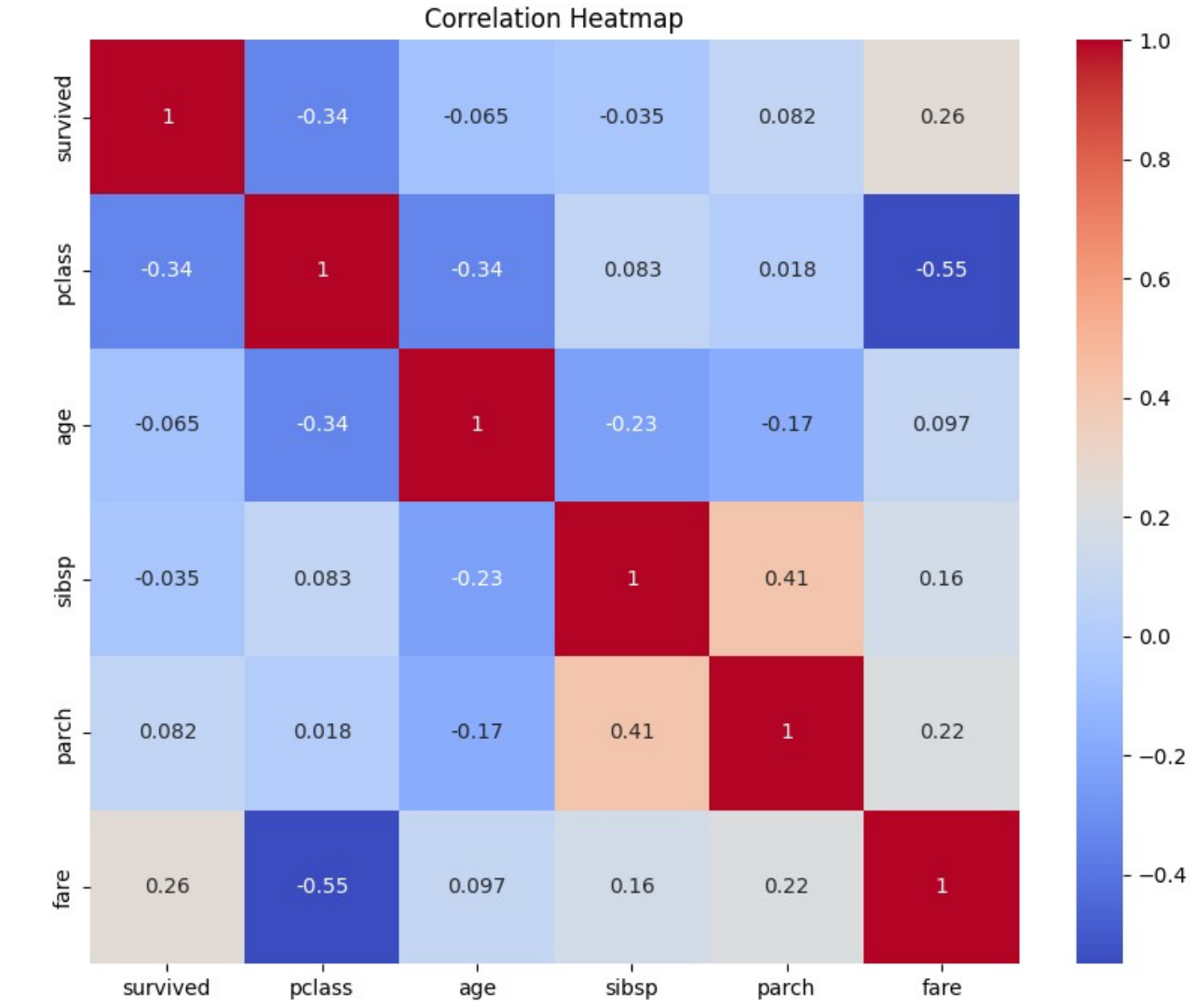

```
plt.figure(figsize=(10, 8))

numeric_df = df.select_dtypes(include=['number'])

sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')

plt.title("Correlation Heatmap")

plt.show()
```


Correlation Heatmap
survived
pclass
age
sibsp
parch
fare
1
-0.34
-0.065
-0.035
0.082
0.26
-0.34
1
-0.34
0.083
0.018
-0.55
-0.065
-0.34
1
-0.23
-0.17
0.097
-0.035
0.083
-0.23
1
0.41
0.16
0.082
0.018
-0.17
0.41
1
0.22
0.26
-0.55
0.097
0.16
0.22
1
survived
pclass
age
sibsp
parch
fare
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4